

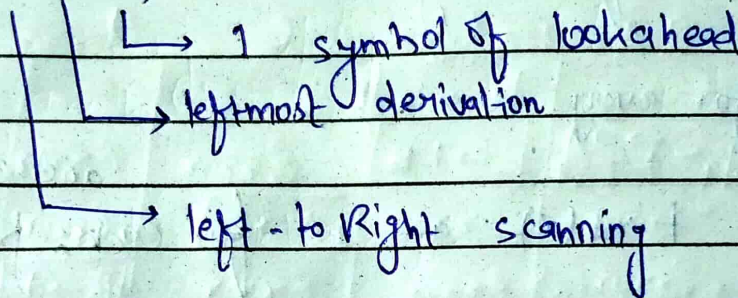
LL(1) :-

→ What is LL(1) :

LL(1) is a types of Top-Down predictive parser.

The name LL(1) has three parts :

LL(1)



→ LL(1) parsing table ?

An LL(1) parsing table tells the parser:

which production rule should I use when I have a particular non-terminal and a particular next input symbol?

The table has:

- Rows → Non-terminals
- Columns → Terminals
- One extra column → \$

→ What do we need before constructing the table?

- ① Grammar
- ② FIRST set for each non-terminals
- ③ FOLLOW set

→ Rules for Constructing LL(1) Table:

suppose we have a production:

$$A \rightarrow \alpha$$

Rule 1: for every terminal in $\text{First}(\alpha)$

for every terminal:

$$\text{First}(\alpha) = a$$

and every
for each production.

put

$$A \rightarrow \alpha$$

in:

$$M[A, a]$$

} Finding FIRST of right
side of production

Rule 2: If ϵ is in $\text{FIRST}(\alpha)$

then find $\text{FOLLOW}(A) = b$

put in $M[A, b]$

• If the production can generate ϵ , use FOLLOW of Non-terminal (left side) to put in ϵ -production

Rule 3: Other cells are errors

If no production is assigned to a table cell:

$$M[A, a] = \text{Error}$$

Eg:

$$S \rightarrow aA \mid bB$$

$$A \rightarrow c$$

$$B \rightarrow d$$

⇒ Finding FIRST :

$$\text{FIRST}(S) = \cancel{\{a, b\}} \{a, b\}$$

$$\text{FIRST}(A) = \{c\}$$

$$\text{FIRST}(B) = \{d\}$$

Finding FOLLOW :

$$\text{FOLLOW}(S) = \{\$ \}$$

$$\text{FOLLOW}(A) = \{\$ \}$$

$$(\text{FOLLOW}(A) = \text{FOLLOW}(S))$$

$$\text{FOLLOW}(B) = \{\$ \}$$

Non-terminal	FIRST	FOLLOW
S	$\{a, b\}$	$\{\$ \}$
A	$\{c\}$	$\{\$ \}$
B	$\{d\}$	$\{\$ \}$

Now constructing the table:

terminals are:

a, b, c, d

and $\$$

	a	b	c	d	$\$$
S	$S \rightarrow aA$	$S \rightarrow bB$	Error	Error	Error
A	Error	Error	$A \rightarrow c$	Error	Error
B	Error	Error	Error	$B \rightarrow d$	Error

Now,

Production 1:

$S \rightarrow aA$

That's our LL(1)
parsing table.

Find:

$FIRST(aA)$

$= a$

put $S \rightarrow aA$
in $M[S, a]$

• Production 2:

$$S \rightarrow bB$$

Find:

FIRST (bB)

$$= b$$

put $S \rightarrow bB$

in $M[S, b]$

• Production 3:

$$A \rightarrow C$$

FIRST (c)

$$= c$$

put $A \rightarrow C$

in $M[A, c]$

• Production 4:

$$B \rightarrow d$$

FIRST (d)

$$= d$$

put

$B \rightarrow d$ in $M[B, d]$

Eg (2)

$S \rightarrow aA$

$A \rightarrow bA \mid \epsilon$

⇒ Finding FIRST :

$FIRST(S) = \{a\}$

$FIRST(A) = \{b, \epsilon\}$

Finding FOLLOW :

$FOLLOW(S) = \{\$ \}$

$FOLLOW(A) = \{\$ \}$

Non-terminal	FIRST	FOLLOW
S	$\{a\}$	$\{\$ \}$
A	$\{b, \epsilon\}$	$\{\$ \}$

Now, Construct table

Terminals :

a, b

and

\$

	a	b	\$
S	$S \rightarrow aA$	Err.	Err.
A	Err.	$A \rightarrow bA$	$A \rightarrow \epsilon$

• Production 1 :

$S \rightarrow aA$

$FIRST(aA)$
 $= a$

put $S \rightarrow aA$
in $M[S, a]$

• Production 2 :

$A \rightarrow bA$

$FIRST(bA)$
 $= b$

That's our LL(1)
parsing table.

put

$$A \rightarrow bA$$

in $M[A, b]$

production 3:

$$A \rightarrow \epsilon$$

$\epsilon \in \text{FIRST}(\epsilon)$

so we use

$$\text{FOLLOW}(A) = \{\$ \}$$

put $A \rightarrow \epsilon$

in $M[A, \$]$

Eg ③

$$S \rightarrow AB$$

$$A \rightarrow a | \epsilon$$

$$B \rightarrow b$$

→ Finding FIRST:

$$\text{FIRST}(S) = \{a, b\}$$

$$\text{FIRST}(A) = \{a, \epsilon\}$$

$$\text{FIRST}(B) = \{b\}$$

Finding FOLLOW:

$$\text{FOLLOW}(S) = \{\$ \}$$

$$\text{FOLLOW}(A) = \{b\}$$

$$\text{FOLLOW}(B) = \{\$ \}$$

<u>Non-terminal</u>	<u>FIRST</u>	<u>FOLLOW</u>
S	$\{a, b\}$	$\{\$ \}$
A	$\{a, \epsilon\}$	$\{b\}$
B	$\{b\}$	$\{\$ \}$

Now, Constructing Table:

	a	b	\$
S	$S \rightarrow AB$	$S \rightarrow AB$	Err.
A	$A \rightarrow a$	$A \rightarrow \epsilon$	Err.
B	Err.	$B \rightarrow b$	Err.

that our LL(1)
parsing table.

• Production 1:

$S \rightarrow AB$

$FIRST(AB)$

$= \{a, \epsilon\}$

ϵ
 AB

$FIRST(B) = \{b\}$

$FIRST(AB) = \{a, b\}$

Put $S \rightarrow AB$

in $M[a, a]$ and $M[s, b]$

• Production 2:

$$A \rightarrow a$$

$FIRST(a)$

$$= a$$

put $A \rightarrow a$

in $M[A, a]$

• Production 3:

$$A \rightarrow \epsilon$$

$\epsilon \in FIRST(\epsilon)$

so we use

$$FOLLOW(A) = \{b\}$$

put $A \rightarrow \epsilon$

in $M[A, b]$

• Production 4:

$$B \rightarrow b$$

$$\text{FIRST}(b) = b$$

put $B \rightarrow b$ in

$$M[B, b]$$

→ What if Two production go into the same cell?

Suppose:

$$A \rightarrow aB$$

$$A \rightarrow aC$$

Then:

$$\text{FIRST}(aB) = \{a\}$$

$$\text{FIRST}(aC) = \{a\}$$

Both production wants:

$$M[A, a]$$

we get :

$$M[A, a] = A \rightarrow aB$$

or

$$A \rightarrow aC$$

This is **Conflict**.

$\therefore$ The grammar is **NOT** LL(1).

Eg;

$$S \rightarrow aA \mid aB$$

$$A \rightarrow c$$

$$B \rightarrow d$$

$\Rightarrow$



Non-terminal

FIRST

FOLLOW

S

{a}

{B}

A

{c}

{a}

B

{d}

{a}

Now, we construct table:

	a	c	d	g
S	$S \rightarrow aA / S \rightarrow aB$			
A		$A \rightarrow c$		
B			$B \rightarrow d$	

Production 1:

$S \rightarrow aA$

$FIRST(aA) = a$

$\therefore$ put $S \rightarrow aA$ in $M[S, a]$

Production 2:

$S \rightarrow aB$

$FIRST(aB) = a$

$\therefore$ put $S \rightarrow aB$ in $M[S, a]$

Similar for more productions

At mps, 9]

we have two production

$$S \rightarrow aA$$

$$S \rightarrow aB$$

But LL(1) allows only one production in each table cell.

$\therefore$ Grammar is NOT LL(1).

Q. Construct a predictive parsing table for the following grammar:

$$S \rightarrow AaAb \mid BbBa$$

$$A \rightarrow \epsilon$$

$$B \rightarrow \epsilon$$

$\Rightarrow$

Find FIRST :

$$\text{FIRST}(S) = \{a, b\}$$

$$\text{FIRST}(A) = \{\epsilon\}$$

$$\text{FIRST}(B) = \{\epsilon\}$$

Find FOLLOW :

$$\text{FOLLOW}(S) = \{\$ \}$$

$$\text{FOLLOW}(A) = \{a, b\}$$

$$\text{FOLLOW}(B) = \{a, b\}$$

Non-terminal	FIRST	Follow
S	{a, b}	{ \$ }
A	{ ε }	{ a, b }
B	{ ε }	{ a, b }

~~Parse~~

Now Construct table:

	a	b	\$
S	$S \rightarrow AaAb$	$S \rightarrow BbBa$	Err.
A	$A \rightarrow \epsilon$	$A \rightarrow \epsilon$	Err.
B	$B \rightarrow \epsilon$	$B \rightarrow \epsilon$	Err.

Production 1:

Our LL(1) parse table

$S \rightarrow AaAb$

FIRST(AaAb)

= a

put $S \rightarrow AaAb$

in $M[s, a]$

• Production 2:

$$S \rightarrow BbBa$$

$$\text{FIRST}(BbBa)$$

$$= b$$

put $S \rightarrow BbBa$
in $M[s, b]$

• Production 3:

$$A \rightarrow \epsilon$$

$$\epsilon \in \text{FIRST}(\epsilon)$$

So we use $\text{FOLLOW}(A)$

$$= \{a, b\}$$

put $A \rightarrow \epsilon$
in

$$M[A, a] \text{ \& } M[A, b]$$

• Production 4 :

$$B \rightarrow \epsilon$$

$$\epsilon \in \text{FIRST}(\epsilon)$$

So we use $\text{FOLLOW}(B)$

$$= \{a, b\}$$

$$\text{put } B \rightarrow \epsilon$$

$$\text{in } M[B, a] \text{ and } M[B, b]$$

Hence this grammar is

LL(2) grammar.

→ How to check whether a string is accepted or not using non-recursive predictive parse table.

Consider the grammar:

$$E \rightarrow E + T \mid T$$

$$T \rightarrow T * F \mid F$$

$$F \rightarrow (E) \mid id$$

Suppose we want to check whether this string is accepted:

id + id * id

Step 1: Remove left recursion:

Removing left recursion from E

$$E \rightarrow E + T \mid T$$

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' \mid \epsilon$$

General form:
 $A \rightarrow A\alpha \mid \beta$

becomes:

$$A \rightarrow \beta A'$$

$$A' = \alpha A' \mid \epsilon$$

Removing left recursion from T

$$T \rightarrow T * F \mid F$$

$$T \rightarrow FT'$$

$$T' = *FT' | \epsilon$$

F has no left recursion

Final LL(1) grammar:

$$E \rightarrow TE'$$

$$E' \rightarrow +TE' | \epsilon$$

$$T \rightarrow FT'$$

$$T' = *FT' | \epsilon$$

$$F \rightarrow (E) | id$$

Step 2: Find FIRST and FOLLOW :-

Non-Terminal	FIRST	FOLLOW
E	{ (, id }	{), \$ }
E'	{ +, ϵ }	{), \$ }
T	{ (, id }	{ +,), \$ }
T'	{ *, ϵ }	{ +,), \$ }
F	{ (, id }	{ *, +,), \$ }

Page No. _____

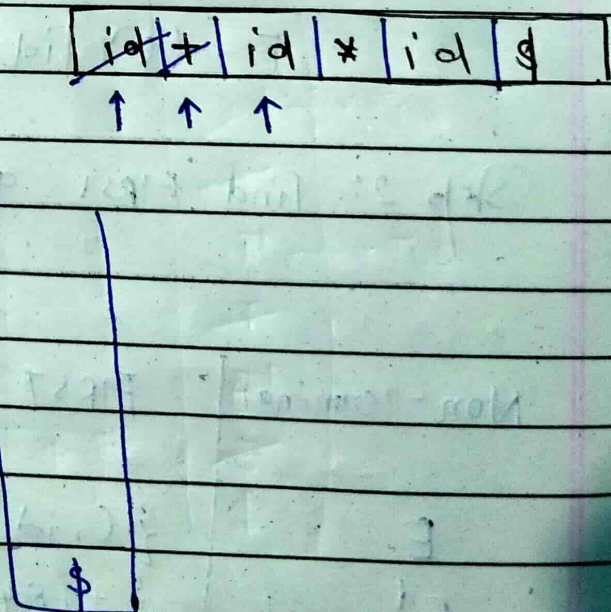
Date

	id	+	*	(	)	\$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \epsilon$	$E' \rightarrow \epsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \epsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \epsilon$	$T' \rightarrow \epsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

Step 4: Use of Stack:

In Back ;

initially



Now we put starting symbol in stack:

which is $\vec{E}$

We look at

$TOS \rightarrow E$

input symbol $\rightarrow id$

Go to parsing table:

$M[E, id]$

F	
T'	
T	$\rightarrow TOS$
E'	
E	$\rightarrow TOS$
$\$$	

Table say:

$E \rightarrow TE'$

So we Remove E and push RHS in Reverse Order

Why Reverse?
Because Stack is LIFO
We want T comes out first

Continue:

$TOS \rightarrow T$

input $\rightarrow id$

$M[T, id]$

$= T \rightarrow FT'$

So remove T and push FT' in Reverse order

Now again

$TOS \rightarrow F$

input $\rightarrow id$

$M[F, id]$

$F \rightarrow id$

So remove F and push id

Now :

$TOS \rightarrow id$

input $\rightarrow id$

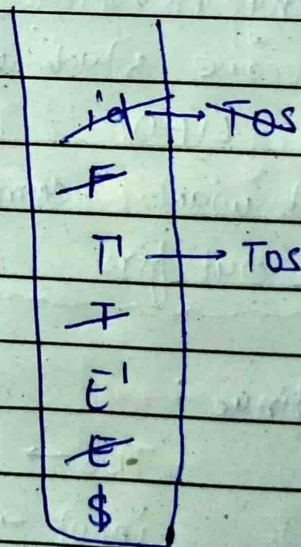
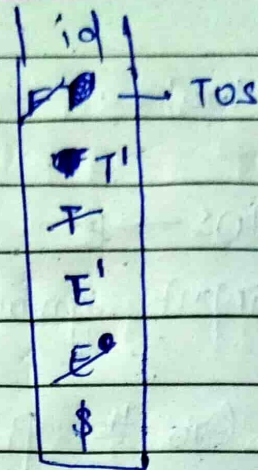
They are same

pop id from the stack and
move input forward.

Again,

$TOS \rightarrow T'$

input $\rightarrow +$



$$M[T', +]$$

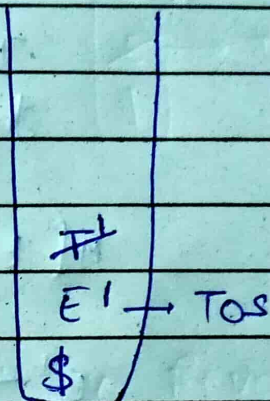
$$T' \rightarrow \epsilon$$

ϵ means nothing to push just remove T'

Now,

$$TOS \rightarrow E'$$

$$\text{input} \rightarrow +$$

$$M[E', +]$$


$$E' \rightarrow +TE'$$

remove E' and push $+TE'$ in reverse order

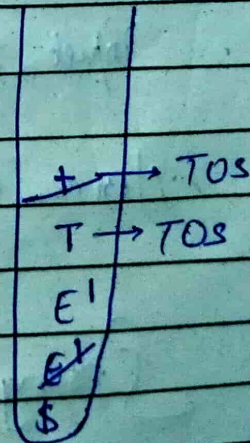
Now,

$$TOS \rightarrow +$$

$$\text{input} \rightarrow +$$

They are same

pop $+$ from the stack and
move input forward



Similarly we do until we are at:

input

id	id	id	id	\$
----	----	----	----	----

↑

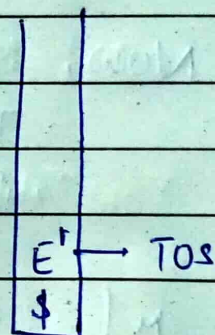
Now,

$TOS \rightarrow E'$

Input $\$$

$M(E', \$)$

$E' \rightarrow \epsilon$

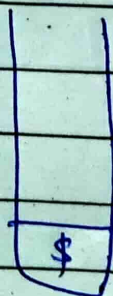


just remove E'

Now,

$TOS \rightarrow \$$

input $\$$



Hence, accepted.

String accepted.

Q. 178

Q. Construct a the non-recursive predictive parse table for the given grammar and check the acceptance of input string

~~abefcg~~ "abefcg" :

$S \rightarrow A$
 $A \rightarrow aB \mid Ad$
 $B \rightarrow bBC \mid F$
 $C \rightarrow cg$

पाना लेना वही तरीका लगेगा जो सबसे पहले वाला प्रश्न में (कैसे थ)

Step ① Remove left Recursion:
 $A \rightarrow aB \mid Ad$ को Remove कर देना

$A \rightarrow aBA'$
 $A' \rightarrow dA' \mid \epsilon$

Step ② Find FIRST, FOLLOW

Step ③ Construct ~~LR~~ LLI table

Step ④ Check or input को बता देना

Accept

(Signature)